CrewCheck project handbook

MVP implementation and review guide

Version 1.4 | 6 November 2026 | Stakeholder review

CrewCheck reviews crew rosters against explicit rules, displays the evidence behind each finding, and preserves the result with the rule version used. This handbook explains the current implementation from first use through technical handover. It accompanies the MVP design review presentation and the working public demo.

The public demo is ready for fictional-data review. Operational use depends on an approved rulebook, authoritative roster and qualification data, assigned employee access, and an agreed internal operating model. PostgreSQL migration remains planned.

Review topics

Requested topic	Handbook section
Proposed user workflow	4 User workflow
How a scheduler uploads a roster	5 Roster upload
How violations are displayed	6 Findings and evidence
How managers view historical flags	7 History and audit review
How threshold values are configured	8 Threshold configuration
Outstanding risks or unresolved questions	18 Decisions and handover

Reading guide

Sections 2–8 cover scope, architecture and user tasks. Sections 9–12 specify data, rules, storage and API behavior. Sections 13–17 cover installation, verification, operations and the internal deployment plan.

This revision includes flexible import choices verified on 6 November 2026: worksheet selection, column mapping, Excel dates and source time zones. The upload screen is current; unchanged result and settings examples were captured on 5 November. Deployment provenance is recorded in outputs/publication.json.

Public demo: https://crewcheck-roster-demo.swoonypopcorn.chatgpt.site

2 Scope and access

The MVP supports CSV and Excel upload, file validation, preview, deterministic checking, per-duty evidence, historical results and versioned thresholds. A human reviews the findings and decides what to do next.

Capability	Scheduler	Manager
Upload and check CSV or XLSX	Yes	Yes
Read completed results	Scheduler-owned records	All records in the same workspace
Read active thresholds	Yes	Yes
Change thresholds and view rule history	No	Yes
View hosted audit events	No	Yes
Edit a roster or resolve a flag in the app	No	No

Deployment distinctions

Environment	Application and data	Identity
Public review demo	Cloudflare Worker and SQLite-based D1	Selectable demo roles in a browser workspace
Local reference	FastAPI and a SQLite file	Two seeded local demo accounts
Planned internal server	Application server and PostgreSQL	Company authentication and assigned roles

The hosted scheduler owner is a demo role within one workspace, not an individual employee account. Managers cannot see other visitors' workspaces. A different browser or device starts a separate workspace. The local reference instead scopes records by user ID.

Excluded from this MVP

PDF, DOCX, legacy .xls and macro-enabled workbooks are outside input scope. The app does not edit schedules, notify crew, assign remediation, approve exceptions or use an LLM. It does not implement a complete aviation regulation, cross-upload duty history, cumulative duty limits or a full qualification registry.

3 System architecture

The public demo serves a browser interface and same-origin API through a Cloudflare Worker. The API validates access, parses files, invokes a pure deterministic checker, and stores records in D1. The local FastAPI reference is a separate implementation, not a service called by the hosted app.

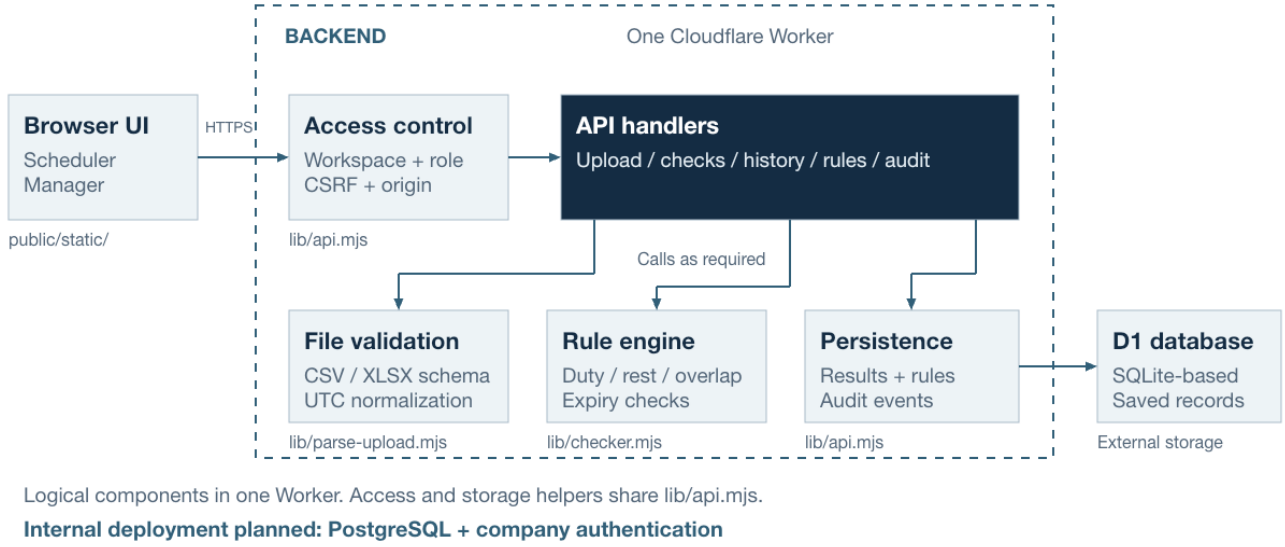


Figure 1 Public demo architecture and planned internal deployment

The dashed backend boundary contains access checks, API handlers, file validation, the rule engine and persistence helpers. These are logical responsibilities within one Worker. Access and D1 helpers share lib/api.mjs. Handlers call the parser for uploads, the checker for Run, and storage for snapshots, history, rules and audit events. They do not form one mandatory pipeline for every request.

Request and persistence lifecycle

1. Select a demo role. The server creates or reuses an isolated browser workspace.
2. Upload a file. The API checks the request, validates all rows and normalizes timestamp offsets to UTC.
3. Save a preview with normalized rows, file SHA-256, ownership and the captured rule version. Discard original file bytes.
4. Run the check explicitly. Load the captured policy, compute findings and save the immutable result with its audit event.
5. Return saved results on later reads. A repeat check for the same upload returns the original result.

The pure checker has no storage or external-service side effects. Upload, rule and result mutations use database batches with their audit entries. A unique upload reference prevents duplicate saved runs.

4 User workflow

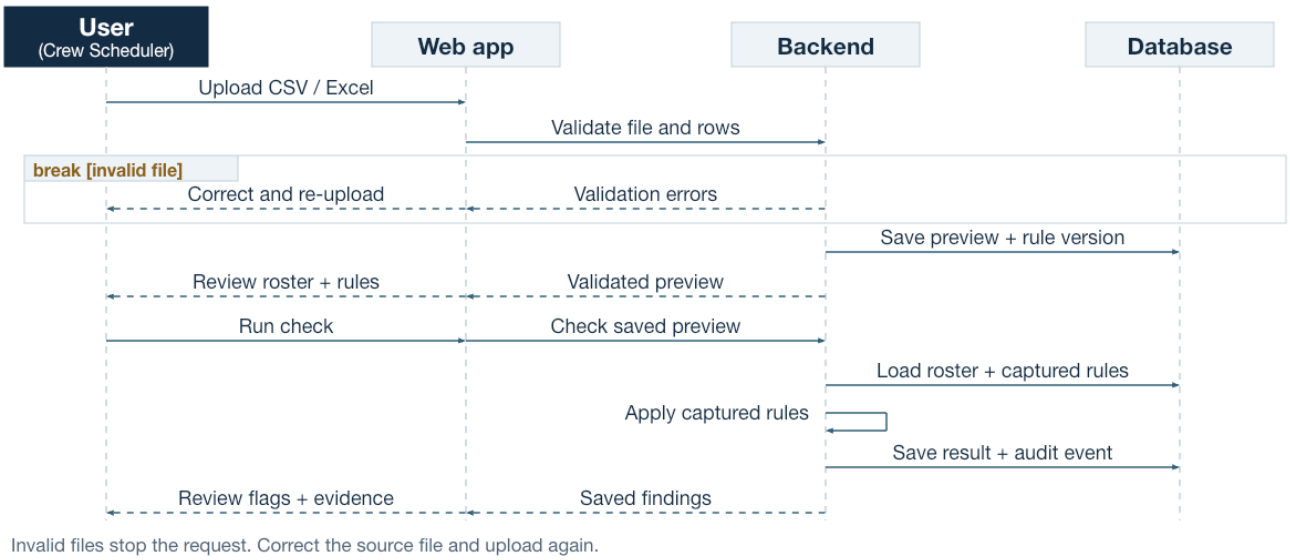


Figure 2 Upload and check sequence

User denotes a person: Crew Scheduler uploads and reviews rosters; Planning Manager reviews history and configures rules. Read the sequence from top to bottom. Solid arrows show actions and requests; dashed arrows show returned data or displayed responses. The invalid-file branch ends the request.

Scheduler journey

Choose Scheduler, upload a roster, resolve any validation errors, review the preview and run the check. Filter findings and inspect Details. Any roster correction happens in the source spreadsheet outside CrewCheck. Upload the corrected file as a new snapshot to compare results.

Manager journey

Choose Manager in the same browser workspace to see both scheduler and manager records. Reopen a saved check, inspect its original evidence, review audit events, or publish a threshold version with a reason. New uploads capture the new rules; existing previews and runs keep their earlier version.

State transitions

Invalid input creates no saved upload or run. An accepted upload becomes a stored preview. A successful check creates one saved result. A preview can remain without a run if the user leaves; there is no draft browser or scheduled draft cleanup. The app provides no acknowledgement or resolution state for findings.

5 Roster upload

1. Open the public demo and select Explore as Scheduler. The starting page is Load a roster.
2. Download CSV or Excel from the starter card. For the meeting, use sample_roster.csv or sample_roster.xlsx.
3. Choose Browse files and select the downloaded file, or drag it into the upload area.
4. Review filename, duty and crew counts, UTC timestamps and rule version. The preview shows up to 20 rows; the checker uses the complete accepted upload.
5. Choose Run roster check. The app saves and displays the result.

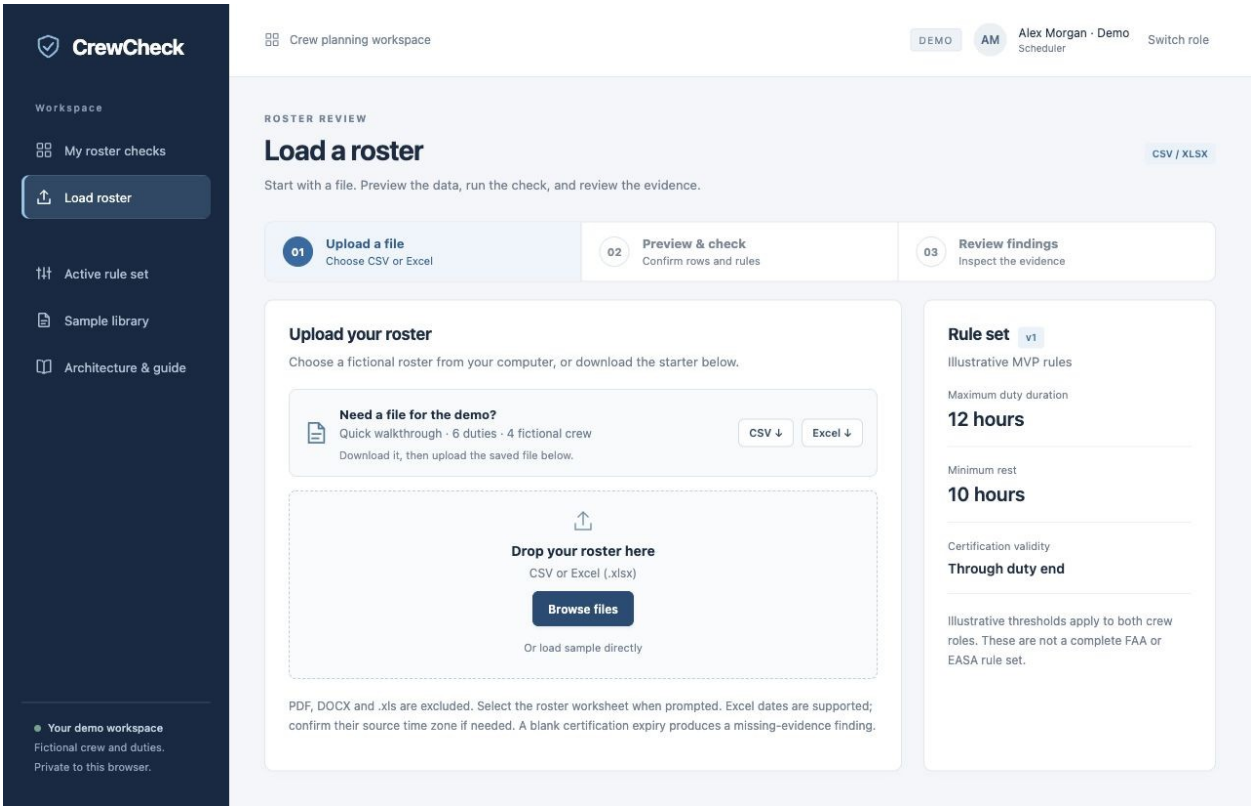


Figure 3 Actual scheduler upload screen

Use the file picker for the meeting demonstration. Or load sample directly is a convenience shortcut. If validation fails, correct the reported rows in the source file and upload again. No partial check runs on an invalid file.

Select the roster worksheet when prompted, match source columns to the required fields, and confirm the source time zone for Excel dates or unzoned timestamps. Review the normalized preview before checking.

6 Findings and evidence

The result page shows duties checked, flagged duties, total findings and crew count. A duty can have multiple findings, so findings and flagged duties are separate measures. Users can filter to flagged duties, search crew or duty ID, and filter by crew role.

Sample duty	Evidence	Initial policy finding
D101	14 hours of duty	Above 12-hour maximum
D103	7 hours between D102 and D103	Below 10-hour minimum rest
D104	Expiry at 12:00 UTC; duty ends at 16:00 UTC	Certification expires before duty end

How to read the details

Open Details on D103. D102 ends on 9 November at 23:00 UTC and D103 starts on 10 November at 06:00 UTC. The calculated interval is seven hours. The drawer compares that value with the ten-hour minimum and identifies REST_MIN and rule version 1.

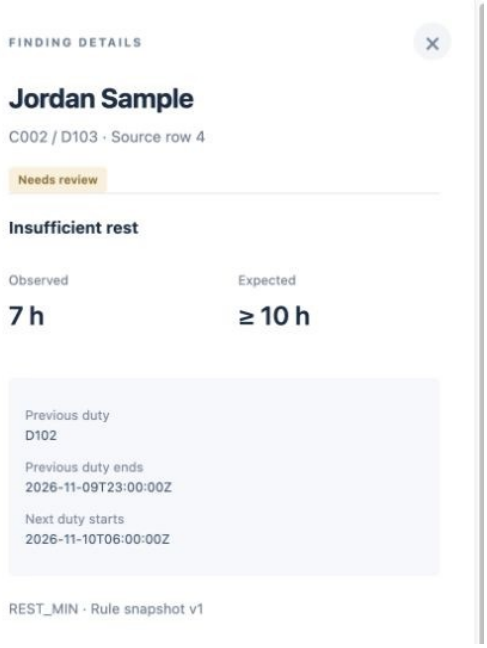


Figure 4 Actual insufficient-rest evidence

The full drawer also identifies the source row, check ID, engine version and source hash. Missing certification is incomplete evidence, not proof of an expired certificate. No flags means only that the configured checks found no issue; first-duty rest and other unimplemented rules remain outside coverage.

7 History and audit review

1. Switch role and choose Explore as Manager in the same browser.
2. Open Check history. Search by filename or filter by check creation date in UTC, then choose Apply filters.
3. Select View check to reopen the original result, captured rules and evidence.
4. Open Audit trail to inspect recorded upload, check, rule-version and demo-role events.

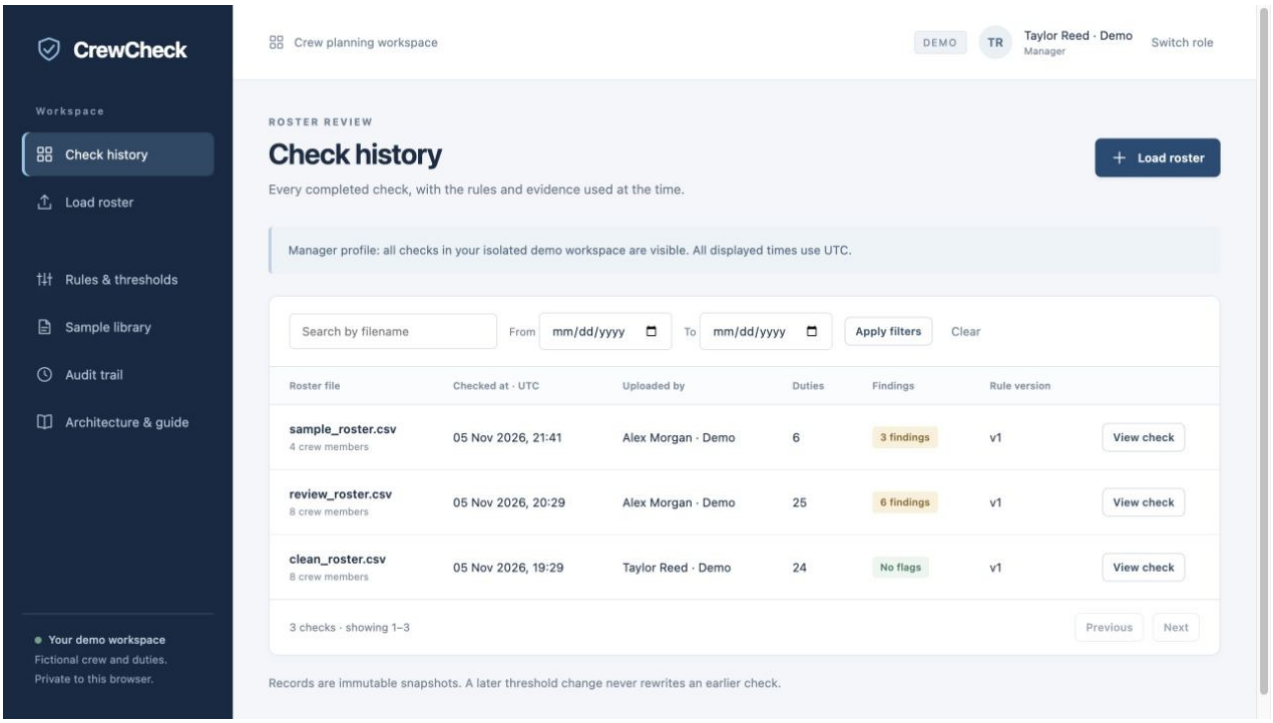


Figure 5 Actual manager history screen

History displays 25 checks per page. Filters use the time the check ran, not the dates of the duties. The manager sees all records in the current visitor workspace. Scheduler access remains limited to scheduler-owned records.

Completed results remain unchanged after threshold updates. Re-uploading the same file produces a new upload and can produce a separate comparison run; repeating Run on one upload returns its existing result.

The audit view shows the latest 100 events. It is an application action history, not an exhaustive log of every view or a tamper-proof store. Demo actor roles are not verified employee identities. Export, acknowledgement, assignment and resolution workflows are not implemented.

8 Threshold configuration

Managers configure a rule name, maximum duty hours, minimum rest hours and whether certification must remain valid through duty start or duty end. A change reason is required. Schedulers can read active settings but cannot publish changes.

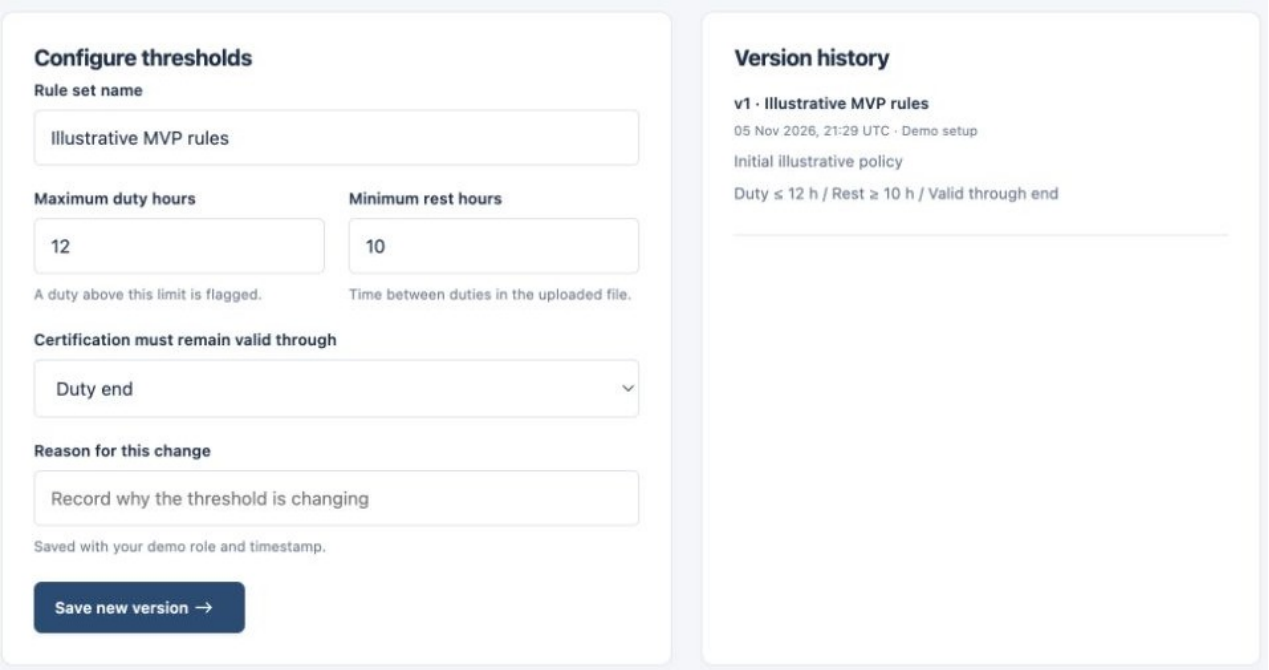


Figure 6 Actual configuration form and version history

Setting	Initial value	Allowed configuration
Maximum duty	12 hours	1–24 hours
Minimum rest	10 hours	1–48 hours
Certification assessment	Duty end	Duty start or duty end
Rule name and change reason	Illustrative MVP rules	Name 3–80 characters; reason 5–400

Save new version records the actor, timestamp and reason. The API checks expected_version and returns a conflict if another change won first. Reload and review the current values before retrying. The hosted demo supports 100 rule versions.

Meeting comparison: set duty to 16 hours, rest to 8 hours and certification to duty start, then upload the same starter again. The new run has one finding; the earlier version 1 run retains three. These are demonstration settings, not approved operational limits.

9 Input contract

Upload UTF-8 CSV or Excel .xlsx. Select a roster worksheet from multi-sheet files. Map different source column names to the six required fields. Additional columns are ignored. There is no fixed row ceiling; blank rows do not count as duties.

Column	Requirement and meaning
crew_id	Required. Stable identifier within the file, for example C001.
crew_name	Required. Consistent for a crew_id within the file.
role	Required. Exactly pilot or cabin.
duty_id	Required. Unique together with crew_id within the file.
duty_start	Required date and time; Excel date cells are supported.
duty_end	Required timestamp strictly after duty_start.
certification_expires_at	Optional column and value. Missing expiry creates a missing-evidence finding.

Timestamp and cell requirements

Use Excel date/time cells or ISO-style text. When a value lacks an offset, choose its source zone, such as UTC or America/New_York. Explicit offsets retain their meaning. Named zones account for clock changes; ambiguous or nonexistent local times require an explicit offset in the affected cell. The preview displays UTC.

Crew name and role must remain consistent within a crew_id. Unmapped notes are ignored. Row numbers refer to the selected source sheet. Selected sheet and time zone remain with saved rows. Missing required values, duplicate duties and invalid periods still require correction.

Deployment capacity

File-size, archive and storage budgets protect the deployment; they are not customer requirements or aviation rules. The file-byte budget is configurable with CREWCHECK_MAX_UPLOAD_BYTES. Current snapshot storage and synchronous processing still have finite capacity. Production volume needs representative files and load testing. Formulas in the selected sheet require values; macros and unsupported archives are not processed.

10 Deterministic check semantics

The checker compares normalized instants using actual elapsed time. It uses the unrounded calculated values for decisions and rounds displayed hour values separately. It sorts duties by start and end for each crew member and tracks the latest-ending prior duty, including nested overlaps.

Rule code	Flag condition	Boundary
DUTY_LIMIT	Duty duration exceeds max_duty_hours	Equality passes
REST_MIN	Non-negative gap below min_rest_hours	Equality passes
DUTY_OVERLAP	Next start precedes latest prior end	No separate rest flag for that gap
CERT_EXPIRED	Supplied expiry precedes the selected assessment instant	Equality passes
CERT_MISSING	Expiry value is blank	Incomplete evidence

Coverage limits

Rest before each crew member’s first duty cannot be assessed without earlier duties in the same file. Stored history is not merged into new checks. The checker has no cumulative duty windows, report-time tables, acclimatization model, augmented-crew rules or approved exception logic.

One expiry field demonstrates evidence handling. It does not establish required licenses, aircraft qualification, recurrent training or revocation. The same illustrative policy applies to pilot and cabin roles. No approved regulatory profile has been implemented.

Reproducibility

A saved result retains normalized rows, findings, summary, coverage note, engine version, file hash and rule snapshot. The same normalized input and policy produce the same findings within the checker version. Historical results are retrieved rather than recalculated. A changed engine requires explicit regression review; old results should remain intact.

Implementation: hosted/lib/checker.mjs and app/engine.py. Shared synthetic fixtures check equivalence for the included scenarios; they do not prove equivalence for every possible input.

11 Storage and audit model

Hosted table	Stored data and purpose
demo_workspaces	Workspace ID, hashed browser capability, expiry, selected role, CSRF token and active rule version.
demo_rule_versions	Workspace and version with immutable policy JSON, reason, actor and time. Unique workspace/version pair.
demo_uploads	Owner role, filename, normalized rows, source SHA-256 and captured rule version.
demo_runs	Saved result JSON, upload reference, owner and creation time. Unique upload_id.
demo_audit_events	Workspace, actor role, action, target, timestamp and contextual details.

Record lifecycle and guarantees

Accepted uploads store normalized content and a hash; original source bytes are discarded. Results and policies have no application update/delete operation. D1 batches commit successful mutations and their corresponding audit events together. Unique run upload references support repeat requests without another result or check event.

Relationships use IDs and application-level workspace checks. The hosted schema has indexes and uniqueness constraints but does not currently declare foreign keys. PostgreSQL migration must add and verify relational constraints rather than assume they already exist.

Recorded actions

demo.created initializes a workspace and its two seed checks. demo.role_selected and demo.role_left record role changes. roster.uploaded records filename, row count, policy, source hash and import choices. roster.checked records the upload, version and finding count. rules.version_created records prior/new versions and the reason. Seed history is identified by the workspace creation event, not individual historical upload/check events.

Workspace access lasts seven days from creation. Automatic record deletion, archival and draft cleanup are not implemented. Application immutability is not protection against a privileged database administrator. The local reference retains uploads, runs and policy versions, but has no equivalent general audit-event table.

12 Hosted API reference

The browser uses same-origin JSON APIs, except multipart file upload. A workspace cookie scopes records. Mutation endpoints other than role selection require X-CSRF-Token from the identity response. The API rejects cross-origin or cross-site writes. Authentication inputs differ in the local reference.

Method and path	Contract
POST /api/auth/login	JSON role: scheduler or manager. Returns demo identity and csrf.
GET /api/auth/me	Read current demo identity.
POST /api/auth/logout	Leave role; retain workspace for later role switching.
POST /api/uploads	Multipart file and optional JSON options for sheet, columns and zone. Returns 201 with preview and rules; 422 import_options requests missing choices.
POST /api/uploads/{id}/check	Run or retrieve the one result for that upload.
GET /api/runs	q, from_date, to_date and offset. Returns items, total, offset and limit 25.
GET /api/runs/{id}	Read authorized complete result snapshot.
GET /api/rules	Read active policy.
GET /api/rules/history	Manager only. Latest 100 policy versions.
POST /api/rules	Manager only. expected_version, name, reason, max_duty_hours, min_rest_hours, certification_at.
GET /api/audit	Manager only. Latest 100 action events.
GET /api/health	Public route with status and storage mode. Not a database read probe.

Errors and caller behavior

401: select a role again. 403: permission, origin or CSRF failure. 404: resource absent or outside access scope. 409: rule version conflict. 413: request exceeds limit. 422: file, field or demo-limit validation failure. 500/503: server or database-binding problem. Errors contain detail; roster errors also contain row/message entries. Import setup responses contain import_options; supply the requested choices before retrying.

GET /api/templates/csv or /xlsx redirects to a sample after access validation. Public /samples/ downloads require no role. No API exists for roster editing, source-file retrieval, finding resolution or bulk audit export.

13 Local reference setup

Use this path to run the retained FastAPI implementation on a local computer. It uses its own SQLite file and demo accounts. It does not reproduce the hosted browser-workspace identity model or hosted audit-event API.

Prerequisites and startup

Python 3.11 or later and uv are required. The lockfile defines Python dependencies. Node is optional for UI syntax checks and diagram regeneration. Run from the project directory:

```
uv sync --frozen
./scripts/run.sh
```

Open http://127.0.0.1:8000. The start script binds to localhost. Open /documentation for the project guide and /openapi.json for the local API schema. Stop the process with Ctrl+C.

Role	Email	Initial demo password
Scheduler	scheduler@crewcheck.local	SchedulerDemo!2026
Manager	manager@crewcheck.local	ManagerDemo!2026

Configuration

Variable	Meaning
PORT	Default 8000; choose another port if occupied.
CREWCHECK_DB	SQLite path. Default data/crewcheck.db.
CREWCHECK_SCHEDULER_PASSWORD	Initial seed password only.
CREWCHECK_MANAGER_PASSWORD	Initial seed password only.
CREWCHECK_SECURE_COOKIE	Set to 1 only with HTTPS. Default local HTTP uses 0.

Export environment values in the shell. The application does not automatically load .env files. Password overrides apply only when accounts are first seeded; they do not rotate existing passwords. These public demo accounts are not suitable for shared production access.

```
PORT=8001 CREWCHECK_DB=data/meeting-demo.db ./scripts/run.sh
```

Use a new database path for an isolated local demonstration instead of deleting prior history. Local sessions last eight hours. Account provisioning, password recovery and company SSO are not implemented.

14 Hosted setup and release

The hosted implementation is in hosted/. Use Node 22.13 or later, npm and the checked-in package-lock.json. The install wrapper handles the project's execution environment. Start from hosted/:

```
npm run install:ci
node --test tests/*.test.mjs
npx tsc --noEmit
npm run build
```

First local D1 initialization

The build generates dist/server/wrangler.json with the DB binding. For a new, empty local D1 state only, apply the initial schema before starting the preview. The following command targets local storage, not the public database:

```
npx wrangler d1 execute DB --local \
  --config dist/server/wrangler.json \
  --persist-to .wrangler/state \
  --file drizzle/0000_absurd_doctor_strange.sql
npm start -- --port 8788
```

Open http://127.0.0.1:8788. Reuse the same state directory on restart. Do not replay the initial SQL on existing tables. The schema filename and command apply to this documented baseline. Later schema evolution requires a tracked migration sequence. Stop a preview before rebuilding its generated assets, then restart it.

Managed public publication

1. Use the existing Sites project identified in .openai/hosting.json and the Sites source/publish workflow.
2. Run tests, type checks and the production build. Prepare the exact source revision and included SQL migrations.
3. Publish through the hosting workflow, which applies production schema changes. Do not run the local initialization command against production.
4. Verify role entry, history, a fictional upload, findings and audit persistence. Record the deployment ID and source commit.

Current publication metadata, source commit and deployment ID are in outputs/publication.json. Keep secrets, .wrangler runtime state, local databases, dependencies and environment files out of source archives. Rollback and schema compatibility require an operator-reviewed plan; this MVP has no automated rollback procedure.

15 Verification and acceptance

Verification on 6 November 2026: 27 local Python tests and 14 hosted tests passed, including import setup, 6,001 parsed duties, a saved 601-duty check and daylight-saving edge cases. TypeScript checks and the hosted production build passed. The local suite emitted a dependency deprecation warning. The Node runner identified node:sqlite as experimental. Neither warning failed a test.

Command	Evidence covered
uv run pytest -q	Local parsing, boundaries, roles, CSRF, versions, repeat checks and restart persistence.
node --test tests/*.test.mjs	Hosted fixture parity, CSV/XLSX agreement, API access, workspace isolation, audit events and rule snapshots.
npx tsc --noEmit	Hosted type checks.
npm run build	Hosted production bundle generation.

Hosted API tests use an in-memory SQLite adapter, not a live D1 service. Passing fixture parity does not prove all-input parity, operational suitability or concurrent production capacity. No load test, PostgreSQL migration test or disaster-recovery rehearsal has been completed.

Repeatable meeting acceptance walkthrough

1. Start with a fresh demo workspace and initial rules. Download and upload sample_roster.csv through Browse files. Confirm six duties and four crew.
2. Run the check. Confirm three findings affecting three duties. Open D103 and verify seven hours of rest against ten hours.
3. Switch to Manager in the same browser. Reopen the scheduler run in history and inspect audit entries.
4. Save a policy version with duty 16, rest 8 and certification at start. Re-upload the starter. Confirm one finding under the new version and three on the old run.
5. Upload input_errors.csv. Select UTC for its unzoned timestamp, then confirm the invalid duration prevents checking.

Fixture pair	Expected with initial rules
sample_roster.csv / .xlsx	6 duties, 4 crew, 3 findings
review_roster.csv / .xlsx	25 duties, 8 crew, 6 findings
clean_roster.csv / .xlsx	24 duties, 8 crew, 0 configured findings
input_errors.csv / .xlsx	Rejected before checking

Actual browser checks covered role selection, file download and upload, preview, findings, evidence, manager history, audit and threshold versioning. These checks used fictional data.

16 Operations and troubleshooting

Current operating boundaries

The public demo isolates visitors by a hashed random capability in an HttpOnly cookie, Secure over HTTPS and SameSite=Lax. Role selection is not employee authentication. A cookie loss or another device means a different workspace. Access expires after seven days without automatic data erasure.

Per-workspace caps are 30 uploads, including two seed checks, and 100 rule versions. The API caps serialized results at 1.5 million bytes. These controls bound normal demo use; they are not a documented global abuse, availability or performance guarantee. No scheduled cleanup, alerting policy or operational service-level target is configured.

Symptom	Action
Missing history	Check browser workspace, selected role and date filters. Dates refer to check creation in UTC.
File rejected	Select the roster sheet, map required fields and supply its source zone. Correct any row errors. Replace formulas in the selected sheet with values.
Session or 403 error	Refresh, choose the role again and retry from the same origin.
409 while saving rules	Reload settings, review the latest version and enter the intended change again.
Old result has old findings	Expected. Upload again to capture a new policy version.
Workspace upload cap	Use another fictional-data browser workspace for a new demo. Existing records remain in their original workspace.
Local port in use	Choose another port; leave unrelated processes running.

Backups and incident handling

For the local reference, use SQLite’s online backup API or stop the app cleanly before copying database state. Do not copy only the main database file while a WAL writer is active. Backup automation, retention, restore testing and hosted database export procedures remain to be assigned and implemented.

For a failed request, record the timestamp, route, non-sensitive error and deployment version. Do not place real roster data or credentials in diagnostic reports. Check the DB binding and server error logs. /api/health reports the route and configured mode; it does not prove database availability. Use a non-sensitive read plus a controlled fictional-data check to verify persistence.

17 PostgreSQL internal deployment plan

The current demo stores audit history in SQLite-based D1. PostgreSQL is the intended internal-server target. There is no implemented PostgreSQL adapter, provisioned internal environment or completed data migration in this baseline.

Planned sequence

1. Confirm the internal application runtime, network boundary, company authentication provider, team permissions and approved rule definitions.
2. Replace visitor workspaces and selectable roles with authenticated organization/user membership. Associate audit records with stable employee actor IDs.
3. Implement a PostgreSQL persistence adapter for upload, policy, result and audit operations. Add migrations, foreign keys, uniqueness constraints and transactions.
4. Map ISO timestamps to timestamptz and structured snapshots to jsonb where appropriate. Preserve record IDs, source hashes and rule references.
5. Restrict the application audit role to required insert/select operations. Define retention, archival, deletion, backups and restore objectives.
6. Re-run parser/engine parity, authorization, conflict and idempotence tests against PostgreSQL. Add concurrency and migration reconciliation checks.
7. Rehearse backup and restore, agree cutover and rollback conditions, then validate the internal pilot with approved fictional or authorized test data.

Migration acceptance criteria

Reconcile IDs, counts, hashes and rule references before and after transfer. Do not recompute historical findings during migration. Demonstrate that old checks remain identical, new rules affect only new uploads, unauthorized users cannot read another team’s records, and repeating a check does not duplicate its result or audit event.

Public demo records are fictional and its actors are unverified roles. Decide whether to discard those demo records from the internal rollout or retain them in an explicitly labeled test dataset. Do not convert demo actor names into asserted employee identities.

Decisions required for operations

Assign ownership for application releases, approved rule changes, data retention, security, backups and support. Determine recovery point and recovery time targets. Evaluate stronger tamper evidence only if the audit policy requires it; application-level append-only behavior alone is insufficient.

18 Decisions and handover

Decision to confirm	Effect on the next phase
Which approved rulebook and version apply to each operation and crew role?	Defines actual limits, terminology, cumulative windows and exceptions.
What do duty start and end include, and are files complete rosters or revisions?	Defines calculations, prior-duty context and duplicate/revision handling.
Which qualification records are authoritative, and which certificates are required?	Replaces the single expiry-field demonstration model.
Which SSO provider, teams, manager provisioning and actual user volumes apply?	Defines identity, authorization and deployment sizing.
What retention, deletion, source-file preservation and recovery requirements apply?	Defines storage lifecycle and operational controls.
Is flag acknowledgement, assignment, remediation or export needed?	Defines whether review evidence becomes a managed case workflow.

Handover package

Provide this PDF and editable Word handbook, the nine-slide MVP design review, the public demo link, the fictional CSV/XLSX pack and the source repository. The project includes architecture/user-flow documentation, Mermaid sources, data contracts, tests, lockfiles and a PostgreSQL transition plan.

Source map

Public UI: hosted/public/static/. Public API and access: hosted/lib/api.mjs. Public parsing: hosted/lib/parse-upload.mjs. Checking: hosted/lib/checker.mjs. Schema: hosted/db/schema.ts and hosted/drizzle/. Tests: hosted/tests/ and tests/. Local reference: app/. Samples: samples/. Release record: outputs/publication.json. Presentation reference: presentation/manifest.json.

Document maintenance

Update this handbook whenever schema, API, access, rule semantics, limits or deployment behavior changes. Record the source revision and verification date. Re-run relevant checks, regenerate the handbook with scripts/build_handbook.py, and inspect the rendered pages. Assign a maintainer during handover; no production service owner has yet been nominated.

Meeting outcome sought: confirm the proposed workflow and identify the rulebook, data contract and internal operating requirements needed for the next implementation phase.